Developing and Evaluating a Predictive Classification Model for Diabetes Diagnosis

Nicole Rodriguez

05/02/25

Contents

Introduction

Diabetes is a chronic health condition that presents significant challenges in both diagnosis and long-term management. Despite its complexity, certain demographic, medical, and clinical indicators can offer valuable insights into an individual's likelihood of developing the disease. This project leverages the Diabetes Dataset available on Kaggle, which provides a collection of patient data including demographic profiles, medical histories, and clinical measurements.

By analyzing these variables, the objective is to identify key predictors of diabetes and build a classification model capable of determining whether an individual is likely to have the condition.

Data

Load Libraries and Packages

The Diabetes Dataset from Kaggle divides the data into the following csv files:

- Training.csv
- Testing.csv

I combined both files for pre-processing and proper splitting later within this analysis.

```
# Load required packages
if (!require("readr"))
  install.packages("readr")
if (!require("tidyverse"))
  install.packages("tidyverse")
if (!require("ggplot2"))
  install.packages("ggplot2")
if (!require("car"))
  install.packages("car")
if (!require("caret"))
  install.packages("caret")

library(readr) # Read data files
library(tidyverse) # Data packages
library(ggplot2)# Data visualization
library(car)# Statistical analysis
library(caret)# Data modeling/training/evaluation
```

Description of Data & Variables

The Diabetes Dataset contains 9 key features that capture a wide range of demographic, medical, and clinical information across a large number of patients. These features include:

- Pregnancies: Total Number of pregnancies
- Glucose: Blood glucose levels
- BloodPressure: Blood pressure readings
- SkinThickness: Thickness of the skin
- Insulin: Blood insulin levels
- BMI: Body mass index
- DiabetesPedigreeFunction: Diabetes percentage
- Age: Individuals age
- Outcome
  - Diabetes diagnosis outcome
    - 0 = NO
    - 1 = YES

The dataset is divided into two parts: a training set with 2,460 observations and a testing set with 308 observations, totaling 2,768 records for analysis.

Load Data

After installing and loading the necessary packages, the two CSV files from the diabetes dataset are imported using **read_csv()** from the *readr* package. To facilitate data analysis and processing, the datasets are then merged using **bind_rows()** from the *dplyr* package.

```
# Load csv files
train_data <- read_csv("Training.csv")
test_data <- read_csv("Testing.csv")
# Combine files
combined_data <- bind_rows(train_data, test_data)
# File dimensions
dim(train_data)
```

```
## [1] 2460    9
```

```
dim(test_data)
```

```
## [1] 308   9
```

```
dim(combined_data)
```

```
## [1] 2768    9
```

```
# View data statistics
glimpse(train_data)
```

```
## Rows: 2,460
## Columns: 9
## $ Pregnancies              <dbl> 6, 1, 8, 1, 0, 5, 3, 10, 2, 8, 4, 10, 10, 1, ~
## $ Glucose                  <dbl> 148, 85, 183, 89, 137, 116, 78, 115, 197, 125~
## $ BloodPressure            <dbl> 72, 66, 64, 66, 40, 74, 50, 0, 70, 96, 92, 74~
## $ SkinThickness            <dbl> 35, 29, 0, 23, 35, 0, 32, 0, 45, 0, 0, 0, 0, ~
## $ Insulin                  <dbl> 0, 0, 0, 94, 168, 0, 88, 0, 543, 0, 0, 0, 0, ~
## $ BMI                      <dbl> 33.6, 26.6, 23.3, 28.1, 43.1, 25.6, 31.0, 35.~
## $ DiabetesPedigreeFunction <dbl> 0.627, 0.351, 0.672, 0.167, 2.288, 0.201, 0.2~
## $ Age                      <dbl> 50, 31, 32, 21, 33, 30, 26, 29, 53, 54, 30, 3~
## $ Outcome                  <dbl> 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, ~
```

```
summary(train_data)
```

```
##   Pregnancies        Glucose      BloodPressure    SkinThickness
##  Min.   : 0.000   Min.   :  0.0   Min.   :  0.00   Min.   : 0.00
##  1st Qu.: 1.000   1st Qu.:100.0   1st Qu.: 64.00   1st Qu.: 0.00
##  Median : 3.000   Median :117.0   Median : 70.00   Median :23.00
##  Mean   : 3.817   Mean   :121.6   Mean   : 68.92   Mean   :20.53
##  3rd Qu.: 6.000   3rd Qu.:142.0   3rd Qu.: 80.00   3rd Qu.:33.00
##  Max.   :17.000   Max.   :197.0   Max.   :122.00   Max.   :63.00
##     Insulin           BMI        DiabetesPedigreeFunction      Age
##  Min.   :  0.00   Min.   : 0.00   Min.   :0.0780           Min.   :21.00
##  1st Qu.:  0.00   1st Qu.:27.10   1st Qu.:0.2517           1st Qu.:24.00
##  Median : 36.00   Median :32.10   Median :0.3810           Median :29.00
##  Mean   : 80.12   Mean   :31.99   Mean   :0.4914           Mean   :32.82
##  3rd Qu.:129.00   3rd Qu.:36.50   3rd Qu.:0.6470           3rd Qu.:39.00
##  Max.   :846.00   Max.   :67.10   Max.   :2.4200           Max.   :81.00
##     Outcome
##  Min.   :0.0000
##  1st Qu.:0.0000
##  Median :0.0000
##  Mean   :0.3902
##  3rd Qu.:1.0000
##  Max.   :1.0000
```

Data Analysis

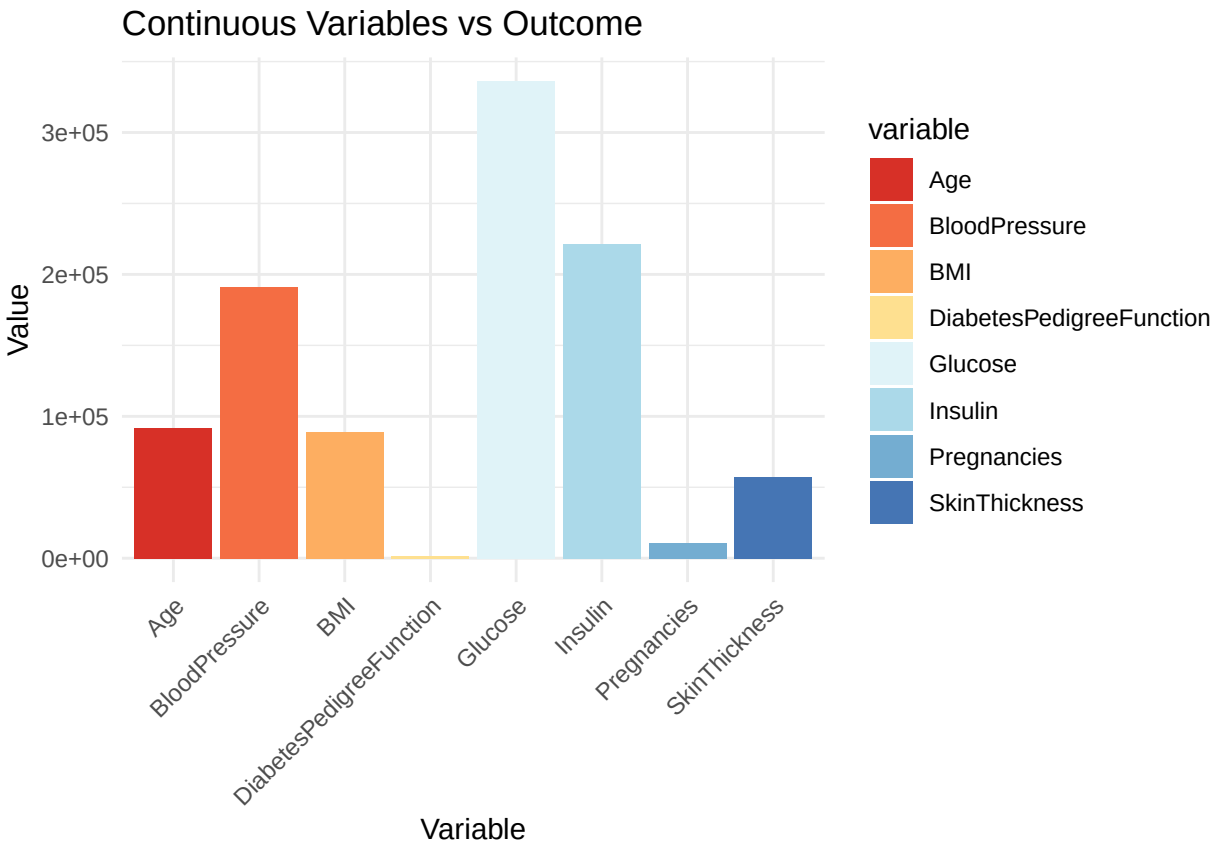
Charts/Graphs

```
# Pivot longer training data for visualization
expand_data <- combined_data %>%
  pivot_longer(cols = -Outcome,
               names_to = "variable", values_to = "value")

# Visualize data in relation to the target variable outcome
ggplot(expand_data, aes(x = variable, y = value, fill = variable)) +
  # Bar plot
```

```r
  geom_col() +
  # Adjust labels
  labs(title = "Continuous Variables vs Outcome", x = "Variable", y = "Value") +
  # Color Palette
  scale_fill_brewer(palette = "RdYlBu") +

  # Add theme
  theme_minimal() +
  # Adjust x-axis variables
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



The barplot above illustrates the distribution of continuous variables in relation to the target variable, **Outcome**. To prepare the data for visualization, the *tidyverse* package was used to reshape the dataset, and *ggplot2* was employed to generate the barplot. The top three continuous variables that show higher values among individuals with a positive diabetes outcome are:

- Glucose
- Insulin
- Blood Pressure

Elevated levels in these variables are commonly associated with an increased risk of having or developing diabetes.

```r
# All variables except outcome
contin_vars <- combined_data %>%
  select(-Outcome)

# Correlation: outcome variable and other variables
outcome_corr <- cor(combined_data$Outcome, contin_vars)
print(outcome_corr)
```

```
##      Pregnancies   Glucose BloodPressure SkinThickness  Insulin       BMI
## [1,]   0.2163369 0.4442624    0.06864028    0.07397502 0.111886 0.3025845
##      DiabetesPedigreeFunction       Age
## [1,]                0.1617524 0.2204091
```

```r
# Positive correlations
positive_corr <- names(outcome_corr[,outcome_corr > 0])

# Moderate positive correlations
moderate_pos_corr <- names(outcome_corr[,outcome_corr > 0.3 & outcome_corr < 0.7])

# Negative correlations
negative_corr <- names(outcome_corr[,outcome_corr < 0])
print(negative_corr)
```

```
## NULL
```

```r
# Results
cat("The following variables had a positive correlation with the outcome of diabetes:\n",
paste("- ", positive_corr, "\n"))
```

```
## The following variables had a positive correlation with the outcome of diabetes:
##  -  Pregnancies
##  -  Glucose
##  -  BloodPressure
##  -  SkinThickness
##  -  Insulin
##  -  BMI
##  -  DiabetesPedigreeFunction
##  -  Age
```

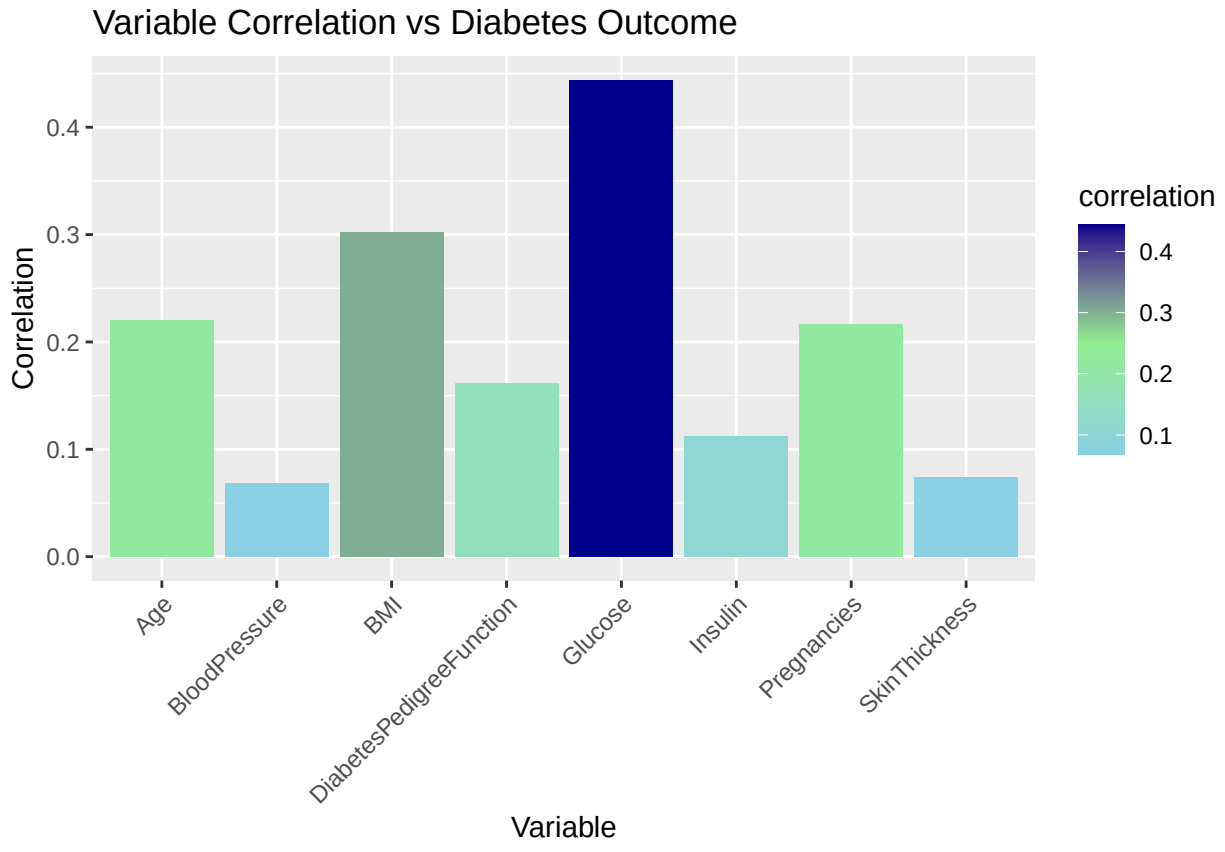
```r
cat("The following variables had a moderate positve correlation with the outcome of diabetes:\n",
paste("- ", moderate_pos_corr, "\n"))
```

```
## The following variables had a moderate positve correlation with the outcome of diabetes:
##  -  Glucose
##  -  BMI
```

```r
# Convert correlation matrix to a data frame
corr_df <- as.data.frame(outcome_corr)
# Obtain rownames
corr_df$variable <- rownames(corr_df)
```

```r
# Reshape df for visualization
corr_df_var <- pivot_longer(corr_df,
                            cols = -variable,
                            names_to = "variable_name",
                            values_to = "correlation")

# Bar plot illustrating outcome correlation with other variables
ggplot(corr_df_var, aes(x = variable_name, y = correlation, fill = correlation)) +
  # Bar plot
  geom_bar(stat = "identity") +
  # Update labels
  labs(title = "Variable Correlation vs Diabetes Outcome", x = "Variable", y = "Correlation") +
  # Adjust x axis text
  theme(axis.text.x = element_text(angle = 45, hjust = 1))+
   scale_fill_gradient2(low = "skyblue", mid = "lightgreen", high = "darkblue", midpoint = 0.25)
```



Correlation values between each feature and the Outcome variable were computed using the cor() function. As shown in the correlation plot above, the four variables with the strongest positive correlation to diabetes diagnosis are:

- Glucose: Correlation: 0.44
- BMI: Correlation: 0.30
- Pregnancies: Correlation: 0.22
- Age: Correlation: 0.22

These results suggest that as the values of these individual variables increase, the likelihood of a diabetes diagnosis also rises. It's important to note that these correlations reflect individual relationships with the outcome and do not account for interactions among variables.

Continuous Feature Outliers

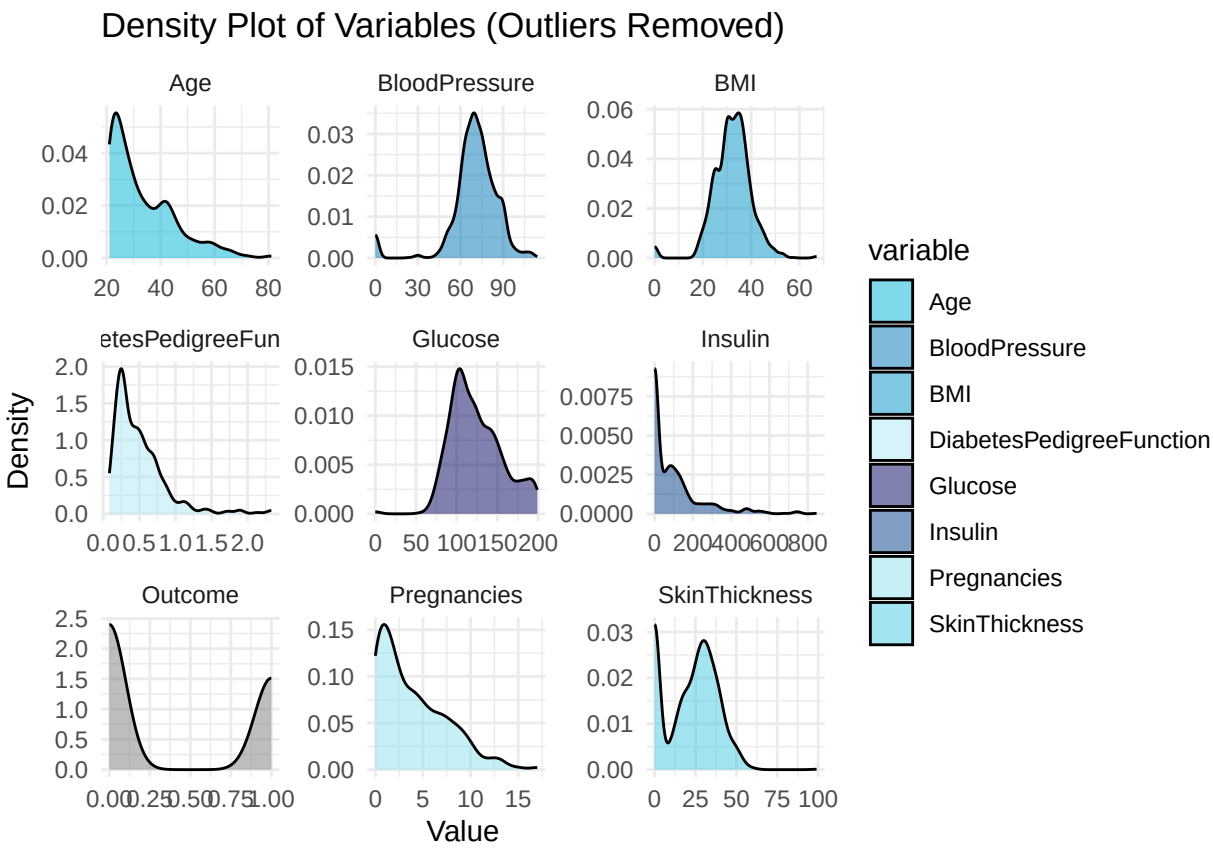
```r
# Detect outliers with IQR
# Calculate IQR
q1 <- apply(contin_vars, 2, quantile, probs = 0.25)
q3 <- apply(contin_vars, 2, quantile, probs = 0.75)
iqr <- q3 - q1

# Identify outliers
lwr <- q1 - 1.5 * iqr
upr <- q3 + 1.5 * iqr
outliers <- combined_data %>%
  filter(across(everything(), ~ . >= lwr & . <= upr))

# Remove outliers
cleaned_data <- combined_data %>%
  anti_join(outliers)

# Reshape for visualization plot
cleaned_data_long <- tidyr::pivot_longer(cleaned_data, everything(),
                                         names_to = "variable",
                                         values_to = "value")

# Visualize with density plot
ggplot(cleaned_data_long, aes(x = value, fill = variable)) +
  # Density plot
  geom_density(alpha = 0.5) +
  # Wrap facets by variable
  facet_wrap(~ variable, scales = "free") +
    # Manual colors
  scale_fill_manual(values = c(
    "Age" = "#00B4D8",
    "BloodPressure" = "#0077B6",
    "BMI" = "#0096C7",
    "DiabetesPedigreeFunction" = "#ADE8F4",
    "Glucose" = "#03045E",
    "Insulin" = "#023E8A",
    "Pregnancies" = "#90E0EF",
    "SkinThickness" = "#48CAE4"
  )) +
  # Update labels
  labs(title = "Density Plot of Variables (Outliers Removed)", x = "Value", y = "Density") +
  # Theme
  theme_minimal()
```



In the density plot shown above, we can see that the removal of outliers changed the density of some of the variables within the data set. We can the distribution of data within each variable more reliably with the removal of these outliers. The outcome variables, is categorical so that's why there's a large, u-shaped dip within it's density plot.

Correlation | Collinearity | Chi-Squared Analysis

```
# All variables except outcome
contin_vars <- cleaned_data %>%
  select(-Outcome)

# Correlation: outcome variable and other variables
outcome_corr <- cor(cleaned_data$Outcome, contin_vars)
print(outcome_corr)
```

```
##      Pregnancies   Glucose BloodPressure SkinThickness   Insulin       BMI
## [1,]   0.2856203 0.5121172     0.1550935    0.07078188 0.1117379 0.2788199
##      DiabetesPedigreeFunction       Age
## [1,]               0.09538894 0.2667333
```

```
# Convert correlation matrix to a data frame
corr_df <- as.data.frame(outcome_corr)

# Obtain rownames
```

```r
corr_df$variable <- rownames(corr_df)

# Reshape df for visualization
updated_corr_df <- pivot_longer(corr_df,
                            cols = -variable,
                            names_to = "variable_name",
                            values_to = "correlation")

# Collinearity with VIF
col_model <- vif(lm(Outcome ~ ., data = cleaned_data))
print(col_model)
```

```
##              Pregnancies                  Glucose            BloodPressure
##                 1.513115                 1.449199                 1.192006
##            SkinThickness                  Insulin                      BMI
##                 1.543338                 1.553497                 1.369307
## DiabetesPedigreeFunction                      Age
##                 1.096557                 1.645004
```

```r
# Chi-squared Analysis
chi_sq <- cleaned_data %>%
  # Exclude outcome variable
  select(-Outcome) %>%
  # Apply to each variable against outcome variable
  map(~chisq.test(.x, cleaned_data$Outcome))
print(chi_sq)
```

```
## $Pregnancies
##
##  Pearson's Chi-squared test
##
## data:  .x and cleaned_data$Outcome
## X-squared = 235.26, df = 16, p-value < 2.2e-16
##
##
## $Glucose
##
##  Pearson's Chi-squared test
##
## data:  .x and cleaned_data$Outcome
## X-squared = 714.19, df = 127, p-value < 2.2e-16
##
##
## $BloodPressure
##
##  Pearson's Chi-squared test
##
## data:  .x and cleaned_data$Outcome
## X-squared = 222.94, df = 42, p-value < 2.2e-16
##
##
## $SkinThickness
```

```
##
##  Pearson's Chi-squared test
##
## data:  .x and cleaned_data$Outcome
## X-squared = 243.36, df = 49, p-value < 2.2e-16
##
##
## $Insulin
##
##  Pearson's Chi-squared test
##
## data:  .x and cleaned_data$Outcome
## X-squared = 627.11, df = 158, p-value < 2.2e-16
##
##
## $BMI
##
##  Pearson's Chi-squared test
##
## data:  .x and cleaned_data$Outcome
## X-squared = 746.49, df = 213, p-value < 2.2e-16
##
##
## $DiabetesPedigreeFunction
##
##  Pearson's Chi-squared test
##
## data:  .x and cleaned_data$Outcome
## X-squared = 1245, df = 389, p-value < 2.2e-16
##
##
## $Age
##
##  Pearson's Chi-squared test
##
## data:  .x and cleaned_data$Outcome
## X-squared = 382.03, df = 50, p-value < 2.2e-16
```

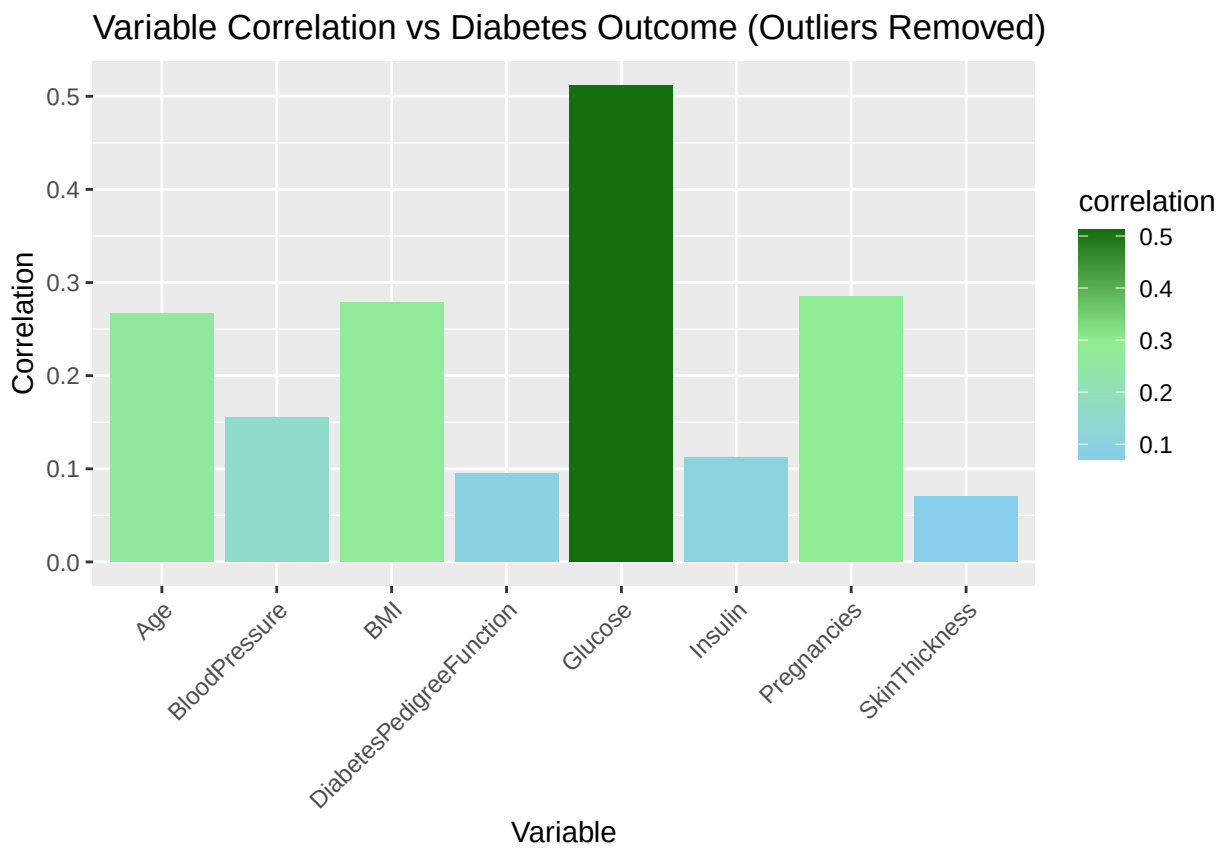
After removing outliers from the dataset, correlation values were recalculated. While the correlation for BMI slightly declined, it still remains an important predictor of the target variable. To further assess multicollinearity among predictors, a **Variance Inflation Factor (VIF)** analysis was conducted. *High* VIF scores typically show collinearity concerns, with values exceeding 5–10 warranting closer inspection. Additionally, a **Chi-Square** test was performed to evaluate the association between each continuous predictor and the outcome variable.

> All predictors returned p-values below 2.2e-16, confirming a statistically significant relationship with the outcome.

Distribution Evaluation

```
# Bar plot: outcome correlation vs other variables
ggplot(updated_corr_df, aes(x = variable_name, y = correlation, fill = correlation)) +
```

```r
  # Bar plot
  geom_bar(stat = "identity") +
  # Update labels
  labs(title = "Variable Correlation vs Diabetes Outcome (Outliers Removed)",
       x = "Variable", y = "Correlation") +
  # Adjust x-axis text
  theme(axis.text.x = element_text(angle = 45, hjust = 1))+
   scale_fill_gradient2(low = "skyblue", mid = "lightgreen", high = "darkgreen", midpoint = 0.3)
```



Re-evaluating the correlation after excluding outliers revealed only minor changes in the barplot. The top four variables (glucose, pregnancies, BMI, and age) remained the most strongly correlated with the outcome variable. Among them, BMI showed a slight drop in its correlation strength.

Data Cleaning & Reshaping

Missing Values & Data Manipulation

```r
# Missing values
missing_vals <- colSums(is.na(cleaned_data))
print(missing_vals)
```

```
##               Pregnancies                   Glucose             BloodPressure
##                         0                         0                         0
##             SkinThickness                   Insulin                       BMI
##                         0                         0                         0
## DiabetesPedigreeFunction                       Age                   Outcome
##                         0                         0                         0
```

As displayed above, the dataset contains no missing values. With this confirmation, we can move forward to the next phase of data cleaning and reshaping to support accurate and consistent analysis.

Normalization/Standardization

```
# Visualize how to impute missing values although there are none
data_imputed <- cleaned_data %>%
  mutate_all(~if_else(is.na(.), mean(., na.rm = TRUE), .))
# Standardize with z-score scale()
standardized_data <- scale(data_imputed)
```

To demonstrate how missing values would be handled, I used the **mutate_all()** function from the dplyr package to scan all columns for missing entries and impute them with their respective column means. After handling missing data, the dataset was standardized using the **scale()** function, which applies z-score normalization to ensure consistency across features.

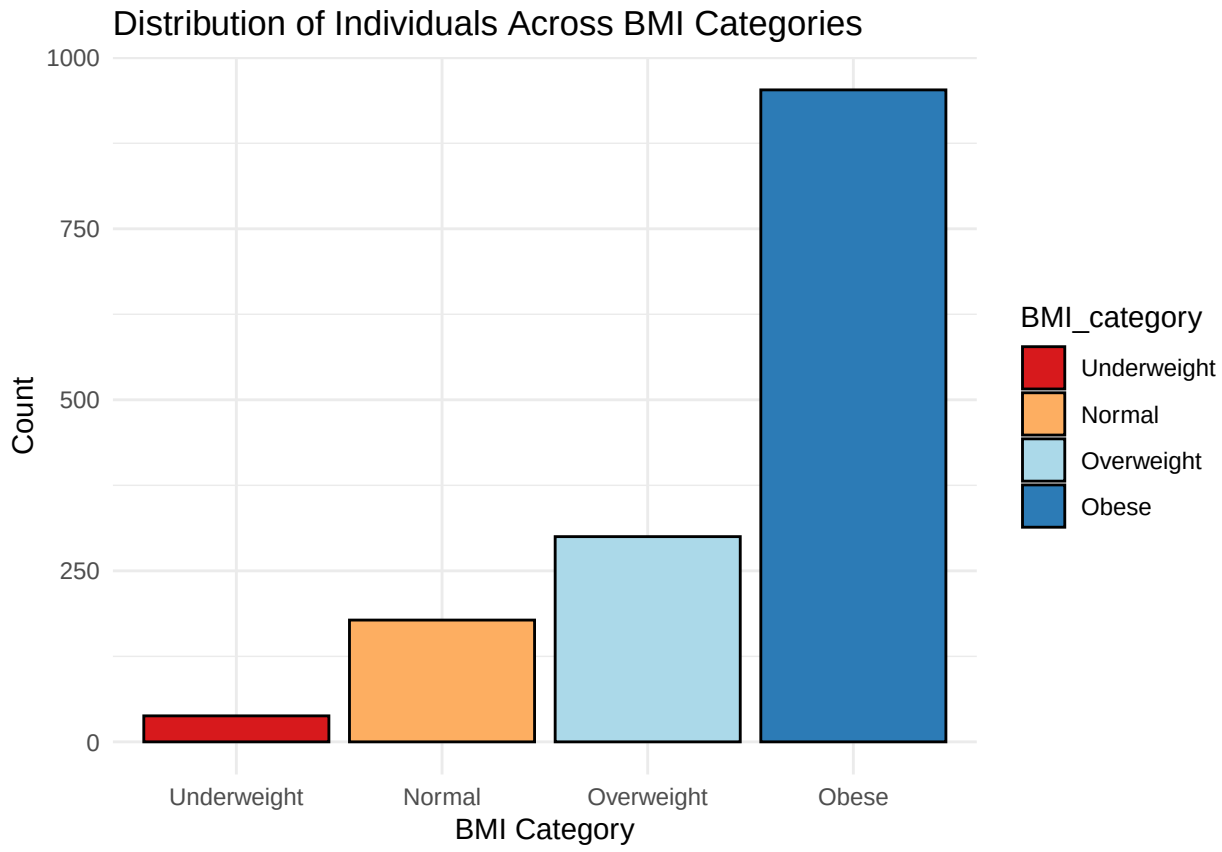
Feature Engineering

```
# Create BMI categories
featured_data <- data_imputed %>%
  mutate(BMI_category = cut(BMI,
                            # Define breaks
                            breaks = c(0, 18.5, 24.9, 29.9, Inf),
                            # Label breaks
                            labels = c("Underweight", "Normal", "Overweight", "Obese"),
                            # Include lowest value in first category
                            include.lowest = TRUE))
# Outcome as factor
featured_data$Outcome <- as.factor(featured_data$Outcome)
head(featured_data)
```

```
## # A tibble: 6 x 10
##   Pregnancies Glucose BloodPressure SkinThickness Insulin   BMI
##         <dbl>   <dbl>         <dbl>         <dbl>   <dbl> <dbl>
## 1           8     183            64             0       0  23.3
## 2           2     197            70            45     543  30.5
## 3           8     125            96             0       0   0
## 4           4     110            92             0       0  37.6
## 5          10     168            74             0       0  38
```

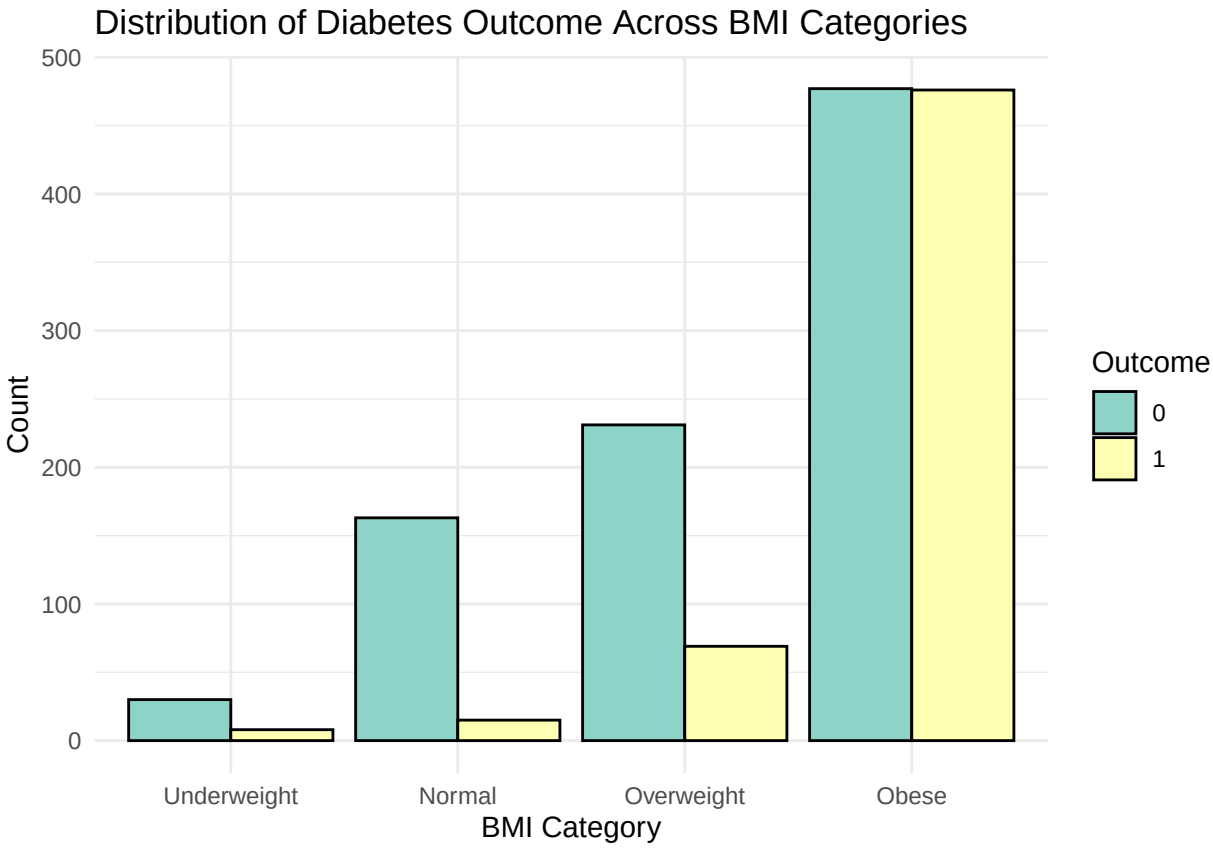
```
## 6           1     189            60            23     846  30.1
## # i 4 more variables: DiabetesPedigreeFunction <dbl>, Age <dbl>, Outcome <fct>,
## #   BMI_category <fct>
```

```r
# Visualize distribution across BMI categories
ggplot(featured_data, aes(x = BMI_category, fill = BMI_category)) +
  # Bar plot
  geom_bar(color = "black") +
  # Color Palette
  scale_fill_brewer(palette = "RdYlBu") +
  # Update labels
  labs(title = "Distribution of Individuals Across BMI Categories",
       x = "BMI Category", y = "Count") +
  # Theme
  theme_minimal()
```




```r
# Visualize BMI category and outcome variable
ggplot(featured_data, aes(x = BMI_category, fill = Outcome)) +
  # Bar plot
  geom_bar(position = "dodge", color = "black") +
  # Color Palette
  scale_fill_brewer(palette = "Set3") +
  # Update labels
  labs(title = "Distribution of Diabetes Outcome Across BMI Categories",
       x = "BMI Category", y = "Count", fill = "Outcome") +
```

```
  # Theme
  theme_minimal()
```



A new variable, **BMI_category**, was introduced by grouping the continuous BMI values into four defined categories: "Underweight," "Normal," "Overweight," and "Obese." This transformation enhances the comprehension of the BMI metric and allows for clearer analysis of its relationship to diabetes risk.

The first bar plot illustrates how individuals are distributed among these BMI categories, with the majority classified as **Obese**, followed by Overweight, Normal, and Underweight, in that order. The second bar plot depicts the prevalence of diabetes within each category, revealing a significantly higher incidence of diabetes among those in the Obese group compared to the others.

Dimensionality Reduction

```
# Principal Component Analysis (PCA)
pca <- prcomp(combined_data[, !names(combined_data) %in% "Outcome"], scale. = TRUE)

# PCA Summary
summary(pca)
```

```
## Importance of components:
##                           PC1    PC2    PC3    PC4     PC5    PC6     PC7
## Standard deviation     1.4621 1.3026 1.0271 0.9119 0.87246 0.8518 0.64533
## Proportion of Variance 0.2672 0.2121 0.1319 0.1040 0.09515 0.0907 0.05206
```

```
## Cumulative Proportion  0.2672 0.4793 0.6112 0.7151 0.81027 0.9010 0.95302
##                            PC8
## Standard deviation     0.61305
## Proportion of Variance 0.04698
## Cumulative Proportion  1.00000
```

```
# Variances by each component
print(pca)
```

```
## Standard deviations (1, .., p=8):
## [1] 1.4620914 1.3026000 1.0270666 0.9119226 0.8724614 0.8518125 0.6453256
## [8] 0.6130535
##
## Rotation (n x k) = (8 x 8):
##                                 PC1         PC2         PC3         PC4
## Pregnancies              -0.1846551 -0.58212275  0.01640920 -0.03541673
## Glucose                  -0.4081471 -0.16500678 -0.40959705 -0.39628033
## BloodPressure            -0.3261912 -0.13557480  0.56668277  0.27019887
## SkinThickness            -0.4263693  0.35806263  0.25506235 -0.02491838
## Insulin                  -0.4483623  0.27463599 -0.24942106 -0.37168543
## BMI                      -0.4380095  0.08229096  0.31958710  0.05819688
## DiabetesPedigreeFunction -0.2655629  0.15783705 -0.52366801  0.78406928
## Age                      -0.2184542 -0.61660947 -0.08485203  0.10852476
##                                 PC5          PC6         PC7         PC8
## Pregnancies               0.510831828 -0.082417001 -0.56101209  0.2074505
## Glucose                  -0.516165031 -0.100915282 -0.27067401 -0.3626379
## BloodPressure            -0.431152239  0.501966697 -0.18087375  0.1024882
## SkinThickness             0.478287556  0.073184841 -0.02816331 -0.6240308
## Insulin                   0.184195646  0.380035812  0.19542997  0.5537484
## BMI                      -0.120990618 -0.755207413  0.17090620  0.2857747
## DiabetesPedigreeFunction  0.008907423 -0.003637674 -0.11833695  0.0384028
## Age                       0.096577207  0.104490527  0.70504403 -0.1891045
```

Model Constuction

To predict whether someone has diabetes based on the dataset's features, three machine learning models will be used:

- Logistic Regression: A simple and effective method for yes/no outcomes. It works well for predicting diabetes by estimating the probability using a logistic curve.
- Random Forest: This model builds many decision trees using the data and combines their results. It's useful for identifying which features play the biggest role in predicting diabetes.
- Support Vector Machine (SVM): Good for two-class problems since SVM finds the best dividing line between diabetic and non-diabetic cases.

Training/Validation Subsets

```r
# Split into training/validation sets - 70/30
# Set seed
set.seed(123)
# Index
index <- createDataPartition(featured_data$Outcome, p = 0.7, list = FALSE)
# Training set
train_data <- featured_data[index, ]
# validation set
valid_data <- featured_data[-index, ]

# Convert Outcome variable to a factor, 1 = YES, 0 = NO diabetes
train_data$Outcome <- factor(train_data$Outcome, levels = c(1, 0))
valid_data$Outcome <- factor(valid_data$Outcome, levels = c(1, 0))
```

The dataset, which includes the new **BMI_category** feature, was split into two parts: 70% for training and 30% for testing.

- Training set (70%): 1029, 10
- Validation set (30%): 440, 10

Model A: Logistic Regression

```r
# Logistic Regression model
model_A <- train(Outcome ~ ., data = train_data, method = "glm", family = "binomial")
```

Model B: Random Forest

```r
# Random forest model
model_B <- train(Outcome ~ ., data = train_data, method = "rf")
```

Model C: Support Vector Machine

```r
# Support vector machine model
model_C <- train(Outcome ~ ., data = train_data, method = "svmRadial")
```

Model Evaluation

Evaluation Metrics & Failure Analysis

```r
# Logistic Regression Confusion matrix
log_conf <- confusionMatrix(predict(model_A, valid_data), valid_data$Outcome)
print(log_conf)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   0
##          1 107  45
##          0  63 225
##
##                Accuracy : 0.7545
##                  95% CI : (0.7116, 0.7941)
##     No Information Rate : 0.6136
##     P-Value [Acc > NIR] : 2.675e-10
##
##                   Kappa : 0.472
##
##  Mcnemar's Test P-Value : 0.1019
##
##             Sensitivity : 0.6294
##             Specificity : 0.8333
##          Pos Pred Value : 0.7039
##          Neg Pred Value : 0.7812
##              Prevalence : 0.3864
##          Detection Rate : 0.2432
##    Detection Prevalence : 0.3455
##       Balanced Accuracy : 0.7314
##
##        'Positive' Class : 1
##
```

The logistic regression model reached an accuracy of 75.45%. Here's how it performed:

- Sensitivity (found diabetic cases correctly): 62.94%
  - 107 true positives
  - 45 false positives
- Specificity (correctly identified non-diabetics): 83.33%
  - 225 true negatives
  - 63 false negatives

The model performs fairly well, but there's room to improve how well it identifies both positive and negative cases.

```
# Random Forest Confusion matrix
rand_conf <- confusionMatrix(predict(model_B, valid_data), valid_data$Outcome)
print(rand_conf)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   0
##          1 158   7
##          0  12 263
```

```
##
##                Accuracy : 0.9568
##                  95% CI : (0.9334, 0.9738)
##     No Information Rate : 0.6136
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.9084
##
##  Mcnemar's Test P-Value : 0.3588
##
##             Sensitivity : 0.9294
##             Specificity : 0.9741
##          Pos Pred Value : 0.9576
##          Neg Pred Value : 0.9564
##              Prevalence : 0.3864
##          Detection Rate : 0.3591
##    Detection Prevalence : 0.3750
##       Balanced Accuracy : 0.9517
##
##        'Positive' Class : 1
##
```

The Random Forest model delivered strong results with an accuracy of 95.68%. Here's how it performed:

- Sensitivity (correctly identified diabetics): 92.94%
  - 158 true positives
  - 7 false positives
- Specificity (correctly identified non-diabetics): 97.41%
  - 263 true negatives
  - 12 false negatives

This model showed excellent overall performance, with high accuracy, precision, and strong ability to detect both diabetic and non-diabetic individuals.

```r
# Support Vector Machine Confusion matrix
cat("Support Vector Machine Confusion Matrix \n\n")
```

```
## Support Vector Machine Confusion Matrix
```

```r
vect_conf <- confusionMatrix(predict(model_C, valid_data), valid_data$Outcome)
print(vect_conf)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   0
##          1 109  24
##          0  61 246
##
```

```
##                Accuracy : 0.8068
##                  95% CI : (0.7668, 0.8427)
##     No Information Rate : 0.6136
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.5755
##
##  Mcnemar's Test P-Value : 9.432e-05
##
##             Sensitivity : 0.6412
##             Specificity : 0.9111
##          Pos Pred Value : 0.8195
##          Neg Pred Value : 0.8013
##              Prevalence : 0.3864
##          Detection Rate : 0.2477
##    Detection Prevalence : 0.3023
##       Balanced Accuracy : 0.7761
##
##        'Positive' Class : 1
##
```

The Support Vector Machine (SVM) model reached an accuracy of 80.68%. Its performance details include:

- Sensitivity (correctly identified diabetics): 64.12%
  - 109 true positives
  - 24 false positives
- Specificity (correctly identified non-diabetics): 91.11%
  - 246 true negatives
  - 61 false negatives

While the SVM model performed well overall, especially in identifying non-diabetic cases, there is room to improve its ability to correctly catch more diabetic cases.

```r
# Extract metrics from Logistic regression model
ma_metrics <- model_A$results
print(ma_metrics)
```

```
##   parameter  Accuracy     Kappa AccuracySD    KappaSD
## 1      none 0.7605951 0.4843562 0.01739062 0.03836992
```

```r
# Extract metrics from random forest model
mb_metrics <- model_B$results
print(mb_metrics)
```

```
##   mtry  Accuracy     Kappa AccuracySD    KappaSD
## 1    2 0.9403118 0.8732225 0.01286199 0.02723380
## 2    6 0.9381228 0.8685966 0.01253549 0.02665234
## 3   11 0.9366350 0.8656627 0.01312137 0.02738563
```

```r
# Extract metrics from support vector machine model
mc_metrics <- model_C$results
print(mc_metrics)
```

```
##        sigma    C  Accuracy     Kappa AccuracySD    KappaSD
## 1 0.09679979 0.25 0.7940393 0.5433155 0.01939523 0.03886350
## 2 0.09679979 0.50 0.8046467 0.5685348 0.01753778 0.03665858
## 3 0.09679979 1.00 0.8211108 0.6073799 0.01803202 0.03475380
```

The following evaluation metrics for each model is defined.

- Logistic Regression
  - Accuracy = 76.06%
  - Kappa = 0.48
  - Accuracy Standard Deviation: 0.02
  - Kappa Standard Deviation: 0.04
- Random Forest
  - Predictors sampled at each split: 2
    - Accuracy = 94.03%
    - Kappa = 0.87
    - Accuracy Standard Deviation: 0.01
    - Kappa Standard Deviation: 0.03
  - Predictors sampled at each split: 6
    - Accuracy = 93.81%
    - Kappa = 0.87
    - Accuracy Standard Deviation: 0.01
    - Kappa Standard Deviation: 0.03
  - Predictors sampled at each split: 11
    - Accuracy = 93.66%
    - Kappa = 0.87
    - Accuracy Standard Deviation: 0.01
    - Kappa Standard Deviation: 0.03
- Support Vector Machine
  - Best sigma: 0.0968
  - Cost parameter: 0.25
    - Accuracy = 79.4%
    - Kappa = 0.54
    - Accuracy Standard Deviation: 0.02
    - Kappa Standard Deviation: 0.04
  - Cost parameter: 0.5
    - Accuracy = 80.46%
    - Kappa = 0.57
    - Accuracy Standard Deviation: 0.02
    - Kappa Standard Deviation: 0.04
  - Cost parameter: 1
    - Accuracy = 82.11%
    - Kappa = 0.61
    - Accuracy Standard Deviation: 0.02
    - Kappa Standard Deviation: 0.03

Holdout Method Evaluation

```
# Extract logistic regression models performance data
lin_accuracy <- log_conf$overall["Accuracy"] * 100
lin_precision <- log_conf$byClass["Pos Pred Value"]
lin_recall <- log_conf$byClass["Sensitivity"]
lin_f1 <- log_conf$byClass["F1"]

# Results
cat("Holdout Method Evaluation for Logistic Regression Model", "\n")
```

```
## Holdout Method Evaluation for Logistic Regression Model
```

```
cat("Logistic Regression Accuracy:", round(lin_accuracy, 2), "\n")
```

```
## Logistic Regression Accuracy: 75.45
```

```
cat("Logistic Regression Precision:", round(lin_precision, 4), "\n")
```

```
## Logistic Regression Precision: 0.7039
```

```
cat("Logistic Regression Recall:", round(lin_recall, 4), "\n")
```

```
## Logistic Regression Recall: 0.6294
```

```
cat("Logistic Regression F1 Score:", round(lin_f1, 4), "\n\n")
```

```
## Logistic Regression F1 Score: 0.6646
```

```
# Extract Random Forest models performance data
rand_accuracy <- rand_conf$overall["Accuracy"] * 100
rand_precision <- rand_conf$byClass["Pos Pred Value"]
rand_recall <- rand_conf$byClass["Sensitivity"]
rand_f1 <- rand_conf$byClass["F1"]

# Results
cat("Holdout Method Evaluation for Random Forest Model", "\n")
```

```
## Holdout Method Evaluation for Random Forest Model
```

```
cat("Random Forest Accuracy:", round(rand_accuracy, 2), "\n")
```

```
## Random Forest Accuracy: 95.68
```

```
cat("Random Forest Precision:", round(rand_precision, 4), "\n")
```

```
## Random Forest Precision: 0.9576
```

```r
cat("Random Forest Recall:", round(rand_recall, 4), "\n")
```

```
## Random Forest Recall: 0.9294
```

```r
cat("Random Forest F1 Score:", round(rand_f1, 4), "\n\n")
```

```
## Random Forest F1 Score: 0.9433
```

```r
# Extract Support Vector Machine models performance data
vect_accuracy <- vect_conf$overall["Accuracy"] * 100
vect_precision <- vect_conf$byClass["Pos Pred Value"]
vect_recall <- vect_conf$byClass["Sensitivity"]
vect_f1 <- vect_conf$byClass["F1"]

# Results
cat("Holdout Method Evaluation for Support Vector Machine Model", "\n")
```

```
## Holdout Method Evaluation for Support Vector Machine Model
```

```r
cat("Support Vector Machine Accuracy:", round(vect_accuracy, 2), "\n")
```

```
## Support Vector Machine Accuracy: 80.68
```

```r
cat("Support Vector Machine Precision:", round(vect_precision, 4), "\n")
```

```
## Support Vector Machine Precision: 0.8195
```

```r
cat("Support Vector Machine Recall:", round(vect_recall, 4), "\n")
```

```
## Support Vector Machine Recall: 0.6412
```

```r
cat("Support Vector Machine F1 Score:", round(vect_f1, 4), "\n")
```

```
## Support Vector Machine F1 Score: 0.7195
```

Based on the evaluations, the **Random Forest model** stands out with the best performance across all key metrics: **accuracy**, **precision**, **recall**, and **F1 score**. It consistently identifies both diabetic and non-diabetic individuals more effectively than the others. While **Logistic Regression** provides decent results, it falls short compared to the Random Forest model. **Support Vector Machine** performs slightly better than Logistic Regression but still lags behind Random Forest. Overall, **Random Forest** is the most reliable and accurate model for this classification task.

K-Fold Cross-Validation Evaluation

```r
# Define control parameters
ctrl <- trainControl(method = "cv", number = 5)

# Train the logistic regression model with cross-validation
modelA_k <- train(Outcome ~ ., data = train_data, method = "glm",
                  family = "binomial", trControl = ctrl)

# Train the random forest model with cross-validation
modelB_k <- train(Outcome ~ ., data = train_data, method = "rf", trControl = ctrl)

# Train the support vector machine model with cross-validation
modelC_k <- train(Outcome ~ ., data = train_data, method = "svmRadial", trControl = ctrl)

# Summarize the results
summary(modelA_k)
```

```
##
## Call:
## NULL
##
## Coefficients:
##                            Estimate Std. Error z value Pr(>|z|)
## (Intercept)               6.8057923  0.7340873   9.271  < 2e-16 ***
## Pregnancies              -0.0980641  0.0282231  -3.475 0.000512 ***
## Glucose                  -0.0477079  0.0038885 -12.269  < 2e-16 ***
## BloodPressure             0.0051986  0.0050246   1.035 0.300844
## SkinThickness            -0.0021717  0.0062887  -0.345 0.729844
## Insulin                   0.0030413  0.0007109   4.278 1.88e-05 ***
## BMI                      -0.0259996  0.0184473  -1.409 0.158719
## DiabetesPedigreeFunction  0.2152339  0.2305185   0.934 0.350461
## Age                      -0.0153741  0.0087766  -1.752 0.079825 .
## BMI_categoryNormal        2.5553600  0.7244400   3.527 0.000420 ***
## BMI_categoryOverweight    1.3813368  0.6860320   2.014 0.044060 *
## BMI_categoryObese         0.4152868  0.7681749   0.541 0.588773
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1373.28  on 1028  degrees of freedom
## Residual deviance:  916.67  on 1017  degrees of freedom
## AIC: 940.67
##
## Number of Fisher Scoring iterations: 5
```

```r
summary(modelB_k)
```

```
##                 Length Class      Mode
## call               4   -none-     call
## type               1   -none-     character
## predicted       1029   factor     numeric
## err.rate        1500   -none-     numeric
```

```
## confusion          6   -none-     numeric
## votes           2058   matrix     numeric
## oob.times       1029   -none-     numeric
## classes            2   -none-     character
## importance        11   -none-     numeric
## importanceSD       0   -none-     NULL
## localImportance    0   -none-     NULL
## proximity          0   -none-     NULL
## ntree              1   -none-     numeric
## mtry               1   -none-     numeric
## forest            14   -none-     list
## y               1029   factor     numeric
## test               0   -none-     NULL
## inbag              0   -none-     NULL
## xNames            11   -none-     character
## problemType        1   -none-     character
## tuneValue          1   data.frame list
## obsLevels          2   -none-     character
## param              0   -none-     list
```

```
summary(modelC_k)
```

```
## Length  Class   Mode
##      1   ksvm     S4
```

Comparing Models

When comparing accuracy, the **Random Forest** model outperformed both the **Support Vector Machine** and **Logistic Regression** models. It also achieved the highest sensitivity, meaning it was the best at correctly identifying diabetic individuals.

In terms of specificity (correctly identifying non-diabetics), both SVM and Random Forest scored high, while Logistic Regression showed the lowest specificity. Thanks to its high scores in both sensitivity and specificity, the Random Forest model also had the strongest precision.

> Overall, Random Forest delivered the best results across all metrics.

The **SVM** model showed strong *precision* and *sensitivity*, but its accuracy could be improved. Though Logistic Regression had the lowest overall performance, it still offers useful insights into which features are important for diabetes prediction.

Model Tuning & Performance Improvement

Model Tuning/Complexity/Bagging

```r
# Hyperparameter tuning grid - logistic regression
param_A <- expand.grid(
  # Grid for alpha
  alpha = seq(0, 1, by = 0.1),
  # Grid for lambda
  lambda = seq(0, 1, by = 0.1))


# Train the logistic regression model with hyperparameter tuning
tuned_modelA <- train(Outcome ~ ., data = train_data, method = "glmnet",
                      trControl = ctrl,tuneGrid = param_A)

# Hyperparameter tuning grid - random forest
param_B <- expand.grid(
  # Grid for mtry
  mtry = c(2, 4, 6)
)

# Train the random forest model with hyperparameter tuning
tuned_modelB <- train(Outcome ~ ., data = train_data, method = "rf",
                      trControl = ctrl,tuneGrid = param_B)

# Hyperparameter tuning grid - support vector machine
param_C <- expand.grid(
  # Grid for sigma
  sigma = c(0.1, 0.5, 1),
  # Grid for C
  C = c(0.25, 0.5, 1)
)

# Train the support vector machine model with hyperparameter tuning
tuned_modelC <- train(Outcome ~ ., data = train_data, method = "svmRadial",
                      trControl = ctrl,tuneGrid = param_C)
```

```r
# Control parameters for bagging
ctrl_bagging <- trainControl(
  # Repeated cross-validation
  method = "repeatedcv",
  # Folds
  number = 5,
  # Repeats
  repeats = 3,
  # Search method
  search = "grid"
)

# Logistic regression model with bagging
bagging_MA <- train(Outcome ~ ., data = train_data, method = "glm", trControl = ctrl_bagging)

# Random forest model with bagging
bagging_MB <- train(Outcome ~ ., data = train_data, method = "rf", trControl = ctrl_bagging)

# Support vector machine model with bagging
```

```r
bagging_MC <- train(Outcome ~ ., data = train_data, method = "svmRadial", trControl = ctrl_bagging)
```

Heterogeneous Ensemble

```r
# Function to create an ensemble model
create_ensemble <- function(models, data) {
  # Predictions for each model
  predictions <- lapply(models, function(model) predict(model, newdata = data))

  # Combine predictions with simple majority
  ensemble_prediction <- rowSums(do.call(cbind, predictions)) > (length(models) / 2)

  # Binary output
  ensemble_prediction <- as.numeric(ensemble_prediction)

  # Return prediction
  return(ensemble_prediction)
}

# Models
ensemble_models <- list(tuned_modelA, tuned_modelB, tuned_modelC, bagging_MA, bagging_MB, bagging_MC)

# Create the ensemble model
ensemble_prediction <- create_ensemble(ensemble_models, featured_data)

# Evaluate the ensemble model
ensemble_accuracy <- (sum(ensemble_prediction == featured_data$Outcome) / nrow(featured_data)) * 100
print(round(ensemble_accuracy, 2))
```

```
## [1] 38.67
```

Ensemble Application

```r
# Apply the ensemble model to make predictions on valid data
ensemble_prediction <- create_ensemble(ensemble_models, valid_data)

# View head of prediction
head(ensemble_prediction)
```

```
## [1] 1 1 1 1 1 1
```

```r
# Evaluate ensemble model
ensemble_accuracy <- (sum(ensemble_prediction == valid_data$Outcome) / nrow(valid_data)) * 100
print(round(ensemble_accuracy, 2))
```

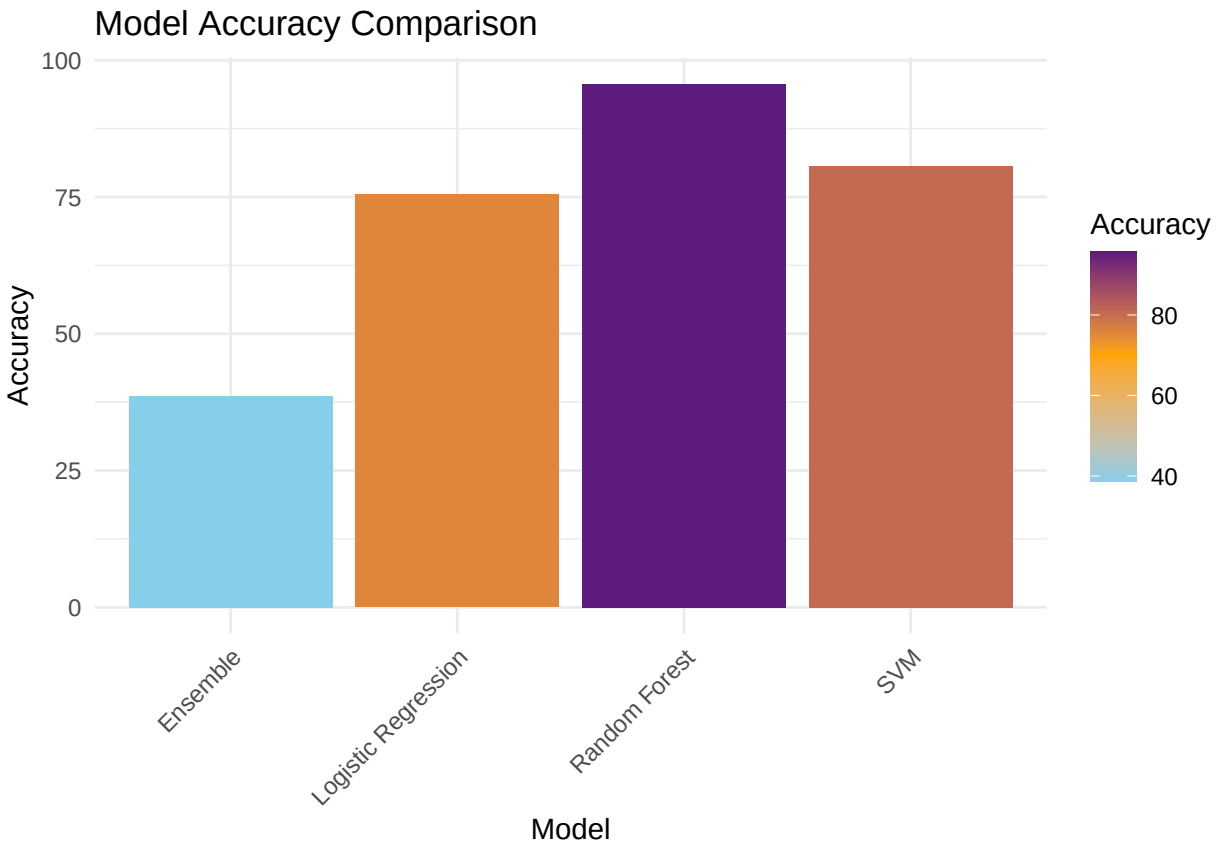
```
## [1] 38.64
```

Comparing Ensemble & Models

```
# Graph the models performances
# Create list of data
models <- c("Logistic Regression", "Random Forest", "SVM", "Ensemble")
accuracy <- c(round(lin_accuracy, 2),
              round(rand_accuracy, 2),
              round(vect_accuracy, 2),
              round(ensemble_accuracy, 2))
# Create data frame with model & accuracy values
accuracy_df <- data.frame(Model = models, Accuracy = accuracy)
# Print data frame
print(accuracy_df)
```

```
##                 Model Accuracy
## 1 Logistic Regression    75.45
## 2       Random Forest    95.68
## 3                 SVM    80.68
## 4            Ensemble    38.64
```

```
# Comparison bar plot
ggplot(accuracy_df, aes(x = Model, y = Accuracy, fill = Accuracy)) +
  # Bar plot
  geom_bar(stat = "identity") +
  # Update labels
  labs(title = "Model Accuracy Comparison",
       x = "Model", y = "Accuracy") +
  # Theme
  theme_minimal() +
  # Adjust x axis text
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
   scale_fill_gradient2(low = "skyblue", mid = "orange", high = "darkblue", midpoint = 70)
```



Interestingly, the **ensemble** model **underperformed** significantly, with an accuracy of only 38.64%, compared to:

- Logistic Regression model's accuracy: 75.45%
- Random Forest model's accuracy: 95.68%
- Support Vector Machine model's accuracy: 80.68%

The ensemble model's performance was less than half that of Logistic Regression, suggesting that combining the models actually weakened their individual predictive power.

Even though each model did well on its own, the ensemble approach was not effective for this dataset and did not enhance prediction accuracy.